

Automata Theory

Practice Problem Set 2

Topics: Pumping Lemma, DFA to Regular Expression, GNFA's, Regular and Context-Free Grammars, Parse Trees and Ambiguity, Pushdown Automata

Difficulty: Easy / Introductory **Number of Questions:** 20

Section A: The Pumping Lemma

Question 1

State the pumping lemma for regular languages. Your statement should clearly identify the pumping length p and the three conditions on the decomposition $s = xyz$ that must hold for every string s in the language with $|s| \geq p$.

Question 2

Use the pumping lemma to prove that the language

$$L = \{ 0^n 1^n \mid n \geq 0 \}$$

over $\Sigma = \{0, 1\}$ is *not* regular.

Hint: Assume L is regular with pumping length p , and consider the string $s = 0^p 1^p$. Show that pumping y produces a string outside L .

Question 3

Use the pumping lemma to show that the language

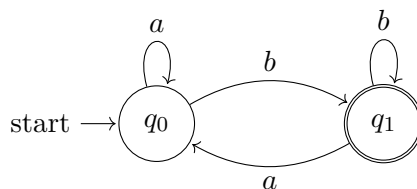
$$L = \{ ww^R \mid w \in \{0, 1\}^* \}$$

of even-length palindromes over $\Sigma = \{0, 1\}$ is *not* regular. (Here w^R denotes the reverse of w .) Use the pumping lemma and choose a suitable string s of length at least p .

Section B: DFA to Regular Expression

Question 4

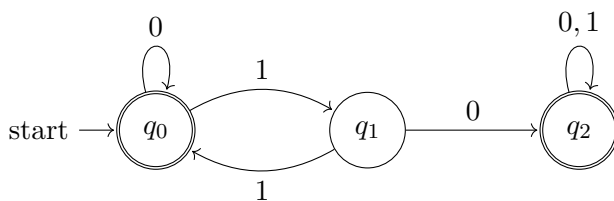
Consider the following DFA over $\Sigma = \{a, b\}$. State q_0 is the start state and q_1 is the only accepting state.



Convert this DFA into an equivalent regular expression. You may use the Arden's method.

Question 5

Consider the following DFA over $\Sigma = \{0, 1\}$. State q_0 is the start state, and q_0 and q_2 are accepting states.



Find a regular expression for the language accepted by this DFA. Then determine which of the strings 0110, 101, and ε are accepted, and justify each answer using your regular expression. Prefer Arden's method.

Section C: Generalized NFAs (GNFAs)

Question 6

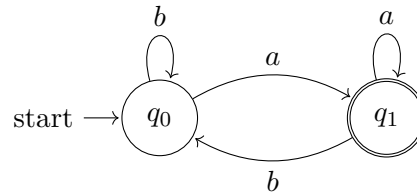
When a GNFA is converted to a regular expression, each internal state is removed one at a time. Suppose that while eliminating a state q_{rip} , we look at a pair of states q_i and q_j connected through it. Let R_1, R_2, R_3, R_4 be the regular-expression labels on the following transitions:

$$q_i \xrightarrow{R_1} q_{\text{rip}}, \quad q_{\text{rip}} \xrightarrow{R_2} q_{\text{rip}} \text{ (self-loop)}, \quad q_{\text{rip}} \xrightarrow{R_3} q_j, \quad q_i \xrightarrow{R_4} q_j.$$

- Write the new regular-expression label on the arrow from q_i to q_j after q_{rip} is removed, in terms of R_1, R_2, R_3, R_4 .
- Why does the standard GNFA state-elimination construction first introduce a single new accept state, distinct from the start state, with an ε -transition from every old accept state?

Question 7

Using the state-elimination (GNFA) method, convert the DFA below over $\Sigma = \{a, b\}$ into an equivalent regular expression. State q_1 is the only accepting state.



First add the standard new start state and new accept state (with ε -transitions into the old start state and out of the old accept state), then eliminate q_0 first and q_1 second. Show the intermediate GNFA after each elimination.

Section D: Regular and Context-Free Grammars**Question 8**

Give a regular (right-linear) grammar that generates the language of all strings over $\Sigma = \{0, 1\}$ that contain an even number of 0's (the number of 1's is arbitrary). For example, ε , 1, 00, and 1010 are in the language, while 0, 10, and 000 are not.

Question 9

Give a context-free grammar that generates the language

$$L = \{0^i 1^j \mid i \geq j \geq 0\},$$

i.e. strings of 0's followed by 1's in which the 0's are at least as many as the 1's. Then give a leftmost derivation of the string 00001.

Question 10

Give a context-free grammar that generates the language of all palindromes over $\Sigma = \{a, b\}$, i.e.

$$L = \{w \in \{a, b\}^* \mid w = w^R\}.$$

Remember to allow both even- and odd-length palindromes, including ε .

Question 11

Give a context-free grammar that generates the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has an equal number of } a\text{'s and } b\text{'s}\}.$$

Note that the a 's and b 's may appear in any order (e.g. $abba$, $baab$, and $aabb$ are all in L). Then give a derivation of the string $abba$.

Question 12

Consider the context-free grammar G with start variable S and rules

$$S \rightarrow aSb \mid SS \mid \varepsilon.$$

- Describe the language $L(G)$ precisely (a characterization in terms of a 's and b 's, not merely a few examples).
- Determine whether the string $aababb$ is in $L(G)$. If it is, give a derivation; if it is not, explain why.

Section E: Parse Trees and Ambiguity**Question 13**

Consider the grammar with start variable S and rules

$$S \rightarrow S + S \mid S \times S \mid a.$$

- Show that this grammar is *ambiguous* by giving two distinct parse trees for the string $a + a \times a$.
- Give an equivalent *unambiguous* grammar for the same language in which $\times$ has higher precedence than $+$ (i.e. enforces the usual precedence).

Question 14

For the grammar

$$S \rightarrow 0S1 \mid 01,$$

draw the parse tree for the string 000111 and state the language generated by the grammar.

Question 15

Consider the grammar with start variable S over $\Sigma = \{a, b\}$:

$$S \rightarrow aS \mid Sa \mid b.$$

- Show that this grammar is ambiguous by exhibiting a string with two distinct leftmost derivations (or two distinct parse trees).
- Describe the language $L(S)$, and give an unambiguous grammar for it.

Section F: Pushdown Automata**Question 16**

Construct a PDA that recognizes the language

$$L = \{a^i b^j \mid i \geq j \geq 0\}$$

over $\Sigma = \{a, b\}$. Draw the state diagram using standard notation, where a transition label $a, b \rightarrow c$ means: read input a , pop b , and push c . Here $a = \varepsilon$ means no input symbol is consumed, $b = \varepsilon$ means no stack symbol is popped, and $c = \varepsilon$ means no stack symbol is pushed. Briefly explain how your PDA handles the “extra” a 's that are not matched by a b .

Question 17

Construct a PDA that recognizes the language

$$L = \{ w \in \{0,1\}^* \mid w \text{ has an equal number of 0's and 1's} \}$$

over $\Sigma = \{0,1\}$. The PDA should accept by final state after the entire input has been consumed. Draw the state diagram using standard notation, and briefly explain the role of the bottom-of-stack marker $\$$ in your design.

Question 18

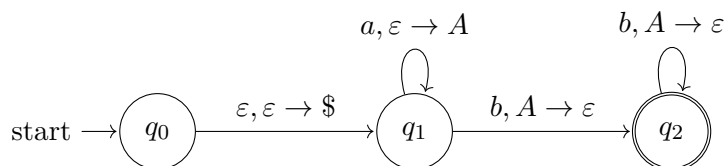
Apply the standard grammar-to-PDA construction to the following context-free grammar with start variable S over $\Sigma = \{a,b\}$:

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon.$$

- Construct the PDA that recognizes $L(S)$. You may draw the state diagram or list the transitions produced by the construction, using $\$$ as the bottom-of-stack marker.
- Describe the language $L(S)$ precisely, and briefly explain why the PDA must be nondeterministic to recognize it.
- Exhibit an *accepting* computation of the PDA on the input $abba$, showing the stack contents (top of stack first) at each step.

Question 19

Consider the PDA below over input alphabet $\Sigma = \{a,b\}$ and stack alphabet $\{A,\$\}$. The start state is q_0 , and q_2 is the only accepting state. Transitions are labeled $a, b \rightarrow c$ (read a , pop b , push c).



The PDA accepts by final state after the entire input has been consumed.

- Trace the computation of the PDA on the input $aabb$, showing the stack contents at each step.
- Identify the context-free language recognized by this PDA, and give a context-free grammar that generates it.

Question 20

Answer each statement as **True** or **False** and briefly justify your answer.

- Every regular language is context-free.
- Every context-free language is regular.
- The pumping lemma for regular languages can be used to prove that a language *is* regular.
- Every DFA can be converted into an equivalent regular expression.
- A pushdown automaton is a finite automaton equipped with a stack.
- The intersection of two context-free languages is always context-free.
- An ambiguous grammar generates a language that has no unambiguous grammar.
- The language $\{0^n 1^n \mid n \geq 0\}$ is context-free but not regular.
- Nondeterministic PDAs are strictly more powerful than deterministic PDAs.
- A regular (right-linear) grammar generates only regular languages.